AUDITORIA BÁSICA

Relatório de Auditoria de Segurança — TaskFlow

30/10/2026 · Projeto: TaskFlow

EXEMPLO FICTÍCIO

Este é um RELATÓRIO DE EXEMPLO. O projeto TaskFlow, os arquivos, as linhas e os achados são fictícios e servem apenas para mostrar o formato do resultado.

Stack detectada

Next.js 14 (App Router) · TypeScript · Supabase (Postgres + Auth + Storage) · Stripe · Docker Compose · GitHub Actions

Escopo auditado

Repositório completo: app/ (34 rotas de API, 21 páginas), components/, lib/, supabase/migrations/ (18 migrations), docker-compose.yml, .github/workflows/ e histórico Git (412 commits).

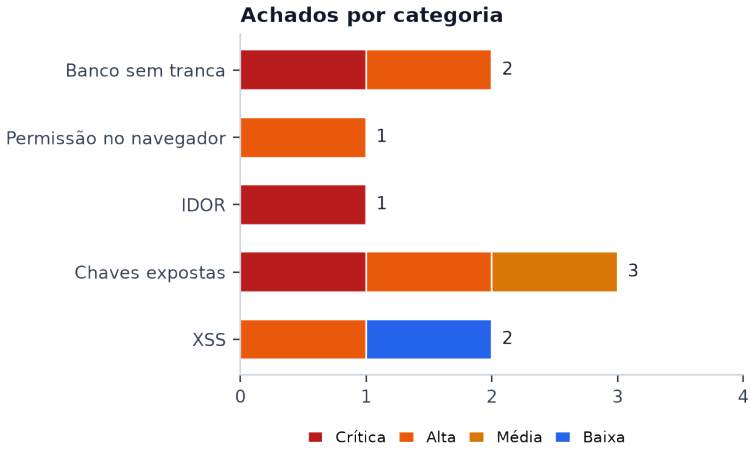
Nota metodológica

Item	Como foi analisado
Banco sem tranca	Mecanismo de isolamento detectado: RLS do Supabase. Verificadas as 18 migrations: tabelas com RLS desabilitado e policies permissivas.
Permissão no navegador	Cruzados os 7 gates de papel do frontend (isAdmin, canManage) com os handlers de app/api/ correspondentes.
IDOR	Percorridos os 34 handlers de rota. Atenção especial aos que usam o client service_role, que ignora RLS.
Chaves expostas	Busca em código, .env*, docker-compose, workflows, bundle de produção (.next/static) e histórico Git.
XSS	Busca por dangerouslySetInnerHTML, renderização de Markdown, href dinâmicos e templates de e-mail.

Resumo executivo

3	4	1	1	0	5
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	PONTOS FORTES





Pontos fortes

Controles encontrados e verificados no código, com evidência.

Controle	Evidência
RLS correto em projects e members	supabase/migrations/20250301_core.sql:40-71: policies filtram por org_id via members em SELECT/INSERT/UPDATE/DELETE.
PATCH e DELETE de tarefas validam dono	app/api/tasks/[id]/route.ts:15 e :48 usam o client do usuário, que respeita RLS.
Rotas de planos exigem admin no servidor	app/api/admin/plans/route.ts:9 verifica role === 'admin' antes de qualquer operação.
Nenhum segredo no código-fonte atual	Varredura em app/, lib/ e components/ sem chaves fixas; .env* está no .gitignore desde o commit 7d03b11.
Escape padrão do React no restante da UI	Os outros 40 componentes que exibem dados de usuário usam JSX ({valor}), que escapa HTML.

Pontos fracos

- Uso do client service_role em rotas de usuário (B-03, B-04): a proteção do RLS fica desligada exatamente onde mais importa.
- Segredos com caminho para o navegador ou para o histórico Git (B-05, B-06).
- RLS inconsistente: tabelas antigas corretas, tabelas novas sem policy ou com USING (true) (B-01, B-02).

Achados detalhados

Banco sem tranca

Severidade	Arquivo:linha	Descrição
CRÍTICA	`supabase/migrations/20250310_invoices.sql:1-14`	**B-01 · Tabela invoices criada sem RLS.** A tabela invoices não tem ENABLE ROW LEVEL SECURITY. Com a anon key (pública no frontend), qualquer pessoa lê e altera faturas de todas as organizações via API REST do Supabase.
ALTA	`supabase/migrations/20250402_tasks_policies.sql:8-10`	**B-02 · Policy de leitura de tasks com USING (true).** RLS está habilitado em tasks, mas a policy de SELECT libera todas as linhas para qualquer usuário autenticado, de qualquer organização.

Permissão no navegador

Severidade	Arquivo:linha	Descrição
ALTA	`app/api/admin/users/[id]/route.ts:8-20`	**B-03 · DELETE de usuário só é bloqueado na tela.** O botão 'Remover usuário' só aparece se isAdmin (components/UserTable.tsx:41), mas o handler DELETE só verifica se há sessão, não o papel.

IDOR

Severidade	Arquivo:linha	Descrição
CRÍTICA	`app/api/projects/[id]/export/route.ts:12-25`	**B-04 · Exportação de projeto sem verificação de dono.** O export usa o client service_role (ignora RLS) e busca o projeto apenas pelo id da URL, sem conferir se o chamador é membro da organização.

Chaves expostas

Severidade	Arquivo:linha	Descrição
CRÍTICA	`lib/supabase/admin.ts:3`	**B-05 · Service role key exposta ao navegador.** A chave service_role é lida de NEXT_PUBLIC_SUPABASE_SERVICE_KEY. Variáveis NEXT_PUBLIC_ vão para o bundle do cliente; a chave foi encontrada em .next/static/chunks/app-3f9a.js.
ALTA	`.env.production (commit a91c2e4, removido em 7d03b11)`	**B-06 · Chave secreta do Stripe no histórico Git.** O arquivo .env.production foi commitado com STRIPE_SECRET_KEY=sk_l**** e removido depois, mas continua acessível no histórico.
MÉDIA	`docker-compose.yml:22`	**B-07 · JWT_SECRET com default público.** Se JWT_SECRET não for definido no deploy, o valor 'changeme' é usado, e não há validação no startup que impeça isso.

XSS

Severidade	Arquivo:linha	Descrição
ALTA	`components/TaskComment.tsx:22`	**B-08 · Comentários em Markdown renderizados sem sanitização.** O comentário do usuário é convertido com marked() e injetado com dangerouslySetInnerHTML. Não há DOMPurify nem similar no projeto.
BAIXA	`lib/email/invite.ts:14`	**B-09 · Nome do convidante sem escape no e-mail de convite.** inviterName é interpolado direto no HTML do e-mail. A maioria dos clientes de e-mail bloqueia scripts, mas permite injeção de links e HTML enganoso.

Detalhamento dos achados

CRÍTICA **B-01 Tabela invoices criada sem RLS**

Categoria: Banco sem tranca

Arquivo	`supabase/migrations/20250310_invoices.sql:1-14`

Evidência	`create table public.invoices (`
	`  id uuid primary key default gen_random_uuid(),`
	`  org_id uuid references orgs(id),`
	`  amount_cents int not null,`
	`  ...`
	`);`
	`-- nenhum 'alter table ... enable row level security'`

Por que é explorável	A tabela invoices não tem ENABLE ROW LEVEL SECURITY. Com a anon key (pública no frontend), qualquer pessoa lê e altera faturas de todas as organizações via API REST do Supabase.
Impacto	Vazamento e adulteração de dados financeiros de todos os clientes.
Correção	Habilitar RLS em invoices e criar policies de SELECT/INSERT/UPDATE/DELETE que exijam org_id pertencente ao usuário (via tabela members).

CRÍTICA **B-04 Exportação de projeto sem verificação de dono**

Categoria: IDOR

Arquivo	`app/api/projects/[id]/export/route.ts:12-25`

Evidência	`const { data } = await supabaseAdmin`
	`  .from('projects')`
	`  .select('*, tasks(*), files(*)')`
	`  .eq('id', params.id)`
	`  .single();`

Por que é explorável	O export usa o client service_role (ignora RLS) e busca o projeto apenas pelo id da URL, sem conferir se o chamador é membro da organização.
Impacto	Trocando o id na URL, um usuário baixa projetos completos (tarefas e arquivos) de outras empresas.
Correção	Usar o client do usuário (respeita RLS) ou adicionar .eq('org_id', membership.org_id) e validar a membership antes.

CRÍTICA **B-05 Service role key exposta ao navegador**

Categoria: Chaves expostas

Arquivo	`lib/supabase/admin.ts:3`

Evidência	`export const supabaseAdmin = createClient(`
	`  process.env.NEXT_PUBLIC_SUPABASE_URL!,`
	`  process.env.NEXT_PUBLIC_SUPABASE_SERVICE_KEY! // eyJh****`
	`);`

Por que é explorável	A chave service_role é lida de NEXT_PUBLIC_SUPABASE_SERVICE_KEY. Variáveis NEXT_PUBLIC_ vão para o bundle do cliente; a chave foi encontrada em .next/static/chunks/app-3f9a.js.
Impacto	Acesso total ao banco, ignorando RLS, para qualquer visitante do site.
Correção	Rotacionar a chave no painel do Supabase, renomear para SUPABASE_SERVICE_ROLE_KEY (sem NEXT_PUBLIC_) e usar só em código de servidor (import 'server-only').

ALTA B-02 Policy de leitura de tasks com USING (true)

Categoria: Banco sem tranca

Arquivo	`supabase/migrations/20250402_tasks_policies.sql:8-10`

Evidência	`create policy "tasks_select" on public.tasks`
	`  for select to authenticated`
	`  using (true);`

Por que é explorável	RLS está habilitado em tasks, mas a policy de SELECT libera todas as linhas para qualquer usuário autenticado, de qualquer organização.
Impacto	Qualquer conta gratuita lê as tarefas de todas as empresas clientes.
Correção	Trocar por using (org_id in (select org_id from members where user_id = auth.uid())).

ALTA B-03 DELETE de usuário só é bloqueado na tela

Categoria: Permissão no navegador

Arquivo	`app/api/admin/users/[id]/route.ts:8-20`

Evidência	`export async function DELETE(req, { params }) {`
	`  const user = await getSessionUser();`
	`  if (!user) return new Response(null, { status: 401 });`
	`  await supabaseAdmin.from('users').delete().eq('id', params.id);`

Por que é explorável	O botão 'Remover usuário' só aparece se isAdmin (components/UserTable.tsx:41), mas o handler DELETE só verifica se há sessão, não o papel.
Impacto	Qualquer usuário logado remove qualquer outro usuário, inclusive administradores.
Correção	Verificar no servidor user.role === 'admin' e se o alvo pertence à mesma organização antes do delete.

ALTA B-06 Chave secreta do Stripe no histórico Git

Categoria: Chaves expostas

Arquivo	`.env.production (commit a91c2e4, removido em 7d03b11)`
Evidência	`STRIPE_SECRET_KEY=sk_l****` `STRIPE_WEBHOOK_SECRET=whse****`
Por que é explorável	O arquivo .env.production foi commitado com STRIPE_SECRET_KEY=sk_l**** e removido depois, mas continua acessível no histórico.
Impacto	Quem tiver acesso ao repositório pode criar cobranças, reembolsos e ler dados de clientes no Stripe.
Correção	Revogar e gerar novas chaves no Stripe. Opcional: limpar o histórico com git filter-repo.

ALTA B-08 Comentários em Markdown renderizados sem sanitização

Categoria: XSS

Arquivo	`components/TaskComment.tsx:22`

Evidência	`<div`
	`  className="prose"`
	`  dangerouslySetInnerHTML={{ __html: marked(comment.body) }}`
	`/>`

Por que é explorável	O comentário do usuário é convertido com marked() e injetado com dangerouslySetInnerHTML. Não há DOMPurify nem similar no projeto.
Impacto	XSS armazenado: um comentário com <img onerror=...> executa script na sessão de quem abrir a tarefa.
Correção	Sanitizar com DOMPurify.sanitize(marked(body)) ou usar react-markdown sem rehype-raw.

MÉDIA B-07 JWT_SECRET com default público

Categoria: Chaves expostas

Arquivo	`docker-compose.yml:22`
Evidência	`environment:` `  - JWT_SECRET=${JWT_SECRET:-changeme}`
Por que é explorável	Se JWT_SECRET não for definido no deploy, o valor 'changeme' é usado, e não há validação no startup que impeça isso.
Impacto	Em deploys sem a variável, um atacante forja tokens de sessão válidos.
Correção	Remover o default e falhar no startup se JWT_SECRET estiver vazio ou com menos de 32 caracteres.

BAIXA B-09 Nome do convidante sem escape no e-mail de convite

Categoria: XSS

Arquivo	`lib/email/invite.ts:14`
Evidência	`html: \` ${inviterName} convidou você para o TaskFlow \``
Por que é explorável	inviterName é interpolado direto no HTML do e-mail. A maioria dos clientes de e-mail bloqueia scripts, mas permite injeção de links e HTML enganoso.
Impacto	Phishing usando o domínio legítimo do TaskFlow.
Correção	Escapar o valor (ex: escape-html) ou usar um template com escape automático (React Email).

Recomendações priorizadas

Prioridade	Ação
P1	Rotacionar a service role key do Supabase e as chaves do Stripe; mover para variáveis sem NEXT_PUBLIC_ (B-05, B-06).
P1	Habilitar RLS em invoices e corrigir a policy de tasks (B-01, B-02).
P1	Adicionar verificação de membership no export de projetos (B-04).
P2	Validar papel admin no servidor em todas as rotas de app/api/admin/ (B-03).
P2	Sanitizar Markdown dos comentários com DOMPurify (B-08).
P3	Remover default de JWT_SECRET e validar no startup; escapar HTML nos e-mails (B-07, B-09).

Issues para o GitHub

Texto pronto para copiar e colar. Nenhuma issue foi aberta automaticamente. Em repositório público, prefira um GitHub Security Advisory privado.

```
--- ISSUE 1 ---

# [Segurança] Tabelas invoices e tasks acessíveis por outras organizações (RLS)

Labels: security, crítica
Achados: B-01, B-02

## Descrição
A tabela invoices não tem RLS habilitado e a policy de leitura de tasks usa USING (true). Com a anon k
ey, que é pública, qualquer pessoa consegue ler dados de todas as organizações.

## Evidência
supabase/migrations/20250310_invoices.sql:1-14
supabase/migrations/20250402_tasks_policies.sql:8-10

## Impacto
Vazamento de faturas e tarefas de todos os clientes.

## Correção sugerida
Criar migration que habilite RLS em invoices e reescreva as policies filtrando por org_id via members.

## Critérios de aceite
- [ ] RLS habilitado em invoices com policies para as 4 operações
- [ ] Policy tasks_select filtra por org_id
- [ ] Teste: usuário da org A não lê dados da org B

--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---

# [Segurança] Service role key do Supabase exposta no bundle do frontend

Labels: security, crítica
Achados: B-05

## Descrição
A chave service_role é lida de uma variável NEXT_PUBLIC_ e acaba no JavaScript entregue ao navegador.

## Evidência
lib/supabase/admin.ts:3
.next/static/chunks/app-3f9a.js

## Impacto
Acesso irrestrito ao banco para qualquer visitante.

## Correção sugerida
Rotacionar a chave, renomear para SUPABASE_SERVICE_ROLE_KEY e importar 'server-only' em lib/supabase/a
dmin.ts.

## Critérios de aceite
- [ ] Chave antiga revogada
- [ ] Nenhuma ocorrência de service key em .next/static após o build
- [ ] Build falha se admin.ts for importado no cliente

--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---

# [Segurança] Exportação de projeto permite baixar dados de outras empresas (IDOR)

Labels: security, crítica
Achados: B-04

## Descrição
GET /api/projects/[id]/export busca o projeto apenas pelo id, usando o client que ignora RLS.

## Evidência
app/api/projects/[id]/export/route.ts:12-25

## Impacto
Exfiltração de projetos completos trocando o id na URL.

## Correção sugerida
Validar a membership do usuário na org do projeto antes da consulta, ou usar o client do usuário.

## Critérios de aceite
- [ ] Retorna 404 para projeto de outra org
- [ ] Teste de regressão cobrindo o caso
- [ ] Demais rotas /api/projects/[id]/* revisadas

--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---

# [Segurança] Chaves do Stripe commitadas no histórico Git

Labels: security, alta
Achados: B-06

## Descrição
O .env.production com STRIPE_SECRET_KEY foi commitado em a91c2e4 e continua no histórico.

## Evidência
.env.production (commit a91c2e4)

## Impacto
Operações financeiras em nome da empresa.

## Correção sugerida
Revogar as chaves no painel do Stripe e gerar novas.

## Critérios de aceite
- [ ] Chaves antigas revogadas no Stripe
- [ ] Novas chaves apenas no provedor de deploy

--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---

# [Segurança] Rota de remoção de usuários sem verificação de admin no servidor

Labels: security, alta
Achados: B-03

## Descrição
A UI esconde o botão para não admins, mas o DELETE /api/admin/users/[id] só exige estar logado.

## Evidência
app/api/admin/users/[id]/route.ts:8-20

## Impacto
Qualquer usuário remove qualquer conta.

## Correção sugerida
Criar helper requireAdmin() e aplicá-lo em todas as rotas de app/api/admin/.

## Critérios de aceite
- [ ] Usuário comum recebe 403
- [ ] Todas as rotas de app/api/admin/ usam requireAdmin()

--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---

# [Segurança] XSS armazenado nos comentários de tarefas

Labels: security, alta
Achados: B-08

## Descrição
Markdown dos comentários é convertido em HTML e injetado sem sanitização.

## Evidência
components/TaskComment.tsx:22

## Impacto
Execução de script na sessão de outros usuários.

## Correção sugerida
Aplicar DOMPurify.sanitize no HTML gerado.

## Critérios de aceite
- [ ] Payload <img src=x onerror=alert(1)> não executa
- [ ] Teste de componente cobrindo o caso

--- FIM ISSUE 6 ---
```

```
--- ISSUE 7 ---

# [Segurança] Endurecimento: default de JWT_SECRET e HTML sem escape em e-mails

Labels: security, média
Achados: B-07, B-09

## Descrição
JWT_SECRET usa 'changeme' como fallback e o e-mail de convite interpola o nome sem escape.

## Evidência
docker-compose.yml:22
lib/email/invite.ts:14

## Impacto
Tokens forjáveis em deploys mal configurados; phishing pelo e-mail de convite.

## Correção sugerida
Remover o fallback e validar no startup; escapar HTML no template.

## Critérios de aceite
- [ ] App não sobe sem JWT_SECRET forte
- [ ] inviterName escapado no e-mail

--- FIM ISSUE 7 ---
```